



Functions & Procedures (Subprograms)

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year I Semester

ACADEMIC YEAR : 2025 - 2026

Functions & Procedures (Subprograms)

Subprograms are named PL/SQL blocks stored in the database. They promote modularity, reuse, and security in database applications. The two main types are Functions and Procedures.

Procedures Overview

- **Purpose:** Performs actions or database modifications (DML).
- **Return value:** Cannot return a value directly in its header. Can return data using OUT parameters.
- **Call style:** Called as a standalone statement:
`execute_payment(acc_id, amount);`
- **Usage:** Standard block for transactional processes (inserts, updates, audits).

Functions Overview

- **Purpose:** Calculates and returns a single data value.
- **Return value:** Must contain a RETURN statement in its header and execution path.
- **Call style:** Called as part of an expression:
`net_sal := salary + calc_bonus(emp_id);`
- **SQL Integration:** Can be called directly inside SQL queries:
`SELECT name, calc_bonus(id) FROM employees;`

Parameter Modes: IN, OUT, and IN OUT

Parameters define how data is passed into and out of stored subprograms:

1. IN Mode (Default)

- Read-Only: The subprogram cannot modify the parameter's value.
- Pass-by-value: Passes a copy of the actual parameter.
- Supports constants, variables, or expressions as input arguments.

2. OUT Mode

- Write-Only: The variable is initialized to NULL inside the block. Used to send data back to caller.
- Must pass a variable, not a literal or expression.
- Subprogram must assign a value before exiting.

3. IN OUT Mode

- Read-Write: Passes an initialized value into the subprogram, modifies it, and returns the updated value to the caller.
- Must pass a variable.
- Ideal for updating records in place.

Pass-by-Value vs. Pass-by-Reference & NOCOPY

Understanding parameter passing mechanics and performance tuning:

Default Pass-by-Value Mechanics

- **Copying Values:** By default, OUT and IN OUT parameters are passed by value. Oracle creates a copy of the argument in memory.
- **Transaction Safety:** If the subprogram fails with an unhandled exception, the copy is discarded and the original variable remains unchanged (clean rollback).
- **Performance Cost:** Copying large collections (e.g. nested tables of 100,000 rows) causes severe CPU and memory overhead.

The NOCOPY Hint (Pass-by-Reference)

- **The Syntax:** Request compiler to pass a pointer directly, skipping the copy:
`PROCEDURE process_data(data IN OUT NOCOPY my_collection);`
- **Performance Boost:** Speeds up execution dramatically when passing large collections, arrays, and objects.
- **The Danger (Side Effect):** Since the subprogram edits the original variable directly, if an error occurs mid-execution, changes remain in the variable (no automatic rollback).

Subprogram Calling Notations

PL/SQL supports multiple syntaxes to bind actual arguments to formal parameters:

Syntax Code Comparison

-- Assume procedure definition:

```
PROCEDURE create_user(p_id NUMBER, p_name VARCHAR2, p_role VARCHAR2 DEFAULT 'Guest');
```

1. Positional Notation (Order based):

```
create_user(101, 'Srinivas', 'Admin'); -- Arguments must match parameter positions exactly.
```

2. Named Notation (Association based):

```
create_user(p_name => 'Srinivas', p_id => 101, p_role => 'Admin'); -- Order does not matter.
```

3. Mixed Notation:

```
create_user(101, p_name => 'Srinivas', p_role => 'Admin'); -- Positional first, then named.
```

4. Default Parameter Omission:

```
create_user(101, 'Srinivas'); -- Omitted parameters resolve to their DEFAULT values.
```

Applications of Subprograms

Subprograms organize logic, secure tables, and improve database performance:

Security & Access Control

Rather than granting users direct read/write permissions on tables, developers grant access to execute stored procedures. This prevents direct table manipulation and SQL injection.

Business Logic Centralization

Keeps business calculations (e.g., tax math, shipping fees) in stored database functions. All client apps (web, mobile, BI) query the same function, preventing logic duplication.

Performance Tuning

Subprograms are compiled once and stored in the Shared Pool memory of Oracle Database. Subsequent calls execute immediately without parsing overhead, boosting speed.

API Layer Integration

Serves as an API layer for applications. Standardizing database interactions inside procedures simplifies schema modifications and system updates.



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management